

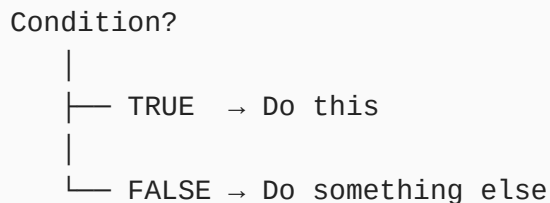
DAY 15

1. `if` Statements

What is an `if` statement?

An `if` statement allows a Bash script to **make decisions based on conditions**.

Think of it as:



For example:

```
if [ "$age" -ge 18 ]
then
    echo "Adult"
fi
```

The script asks:

| Is `age` greater than or equal to 18?

If yes, the command runs.

2. Basic `if` Syntax

The basic structure is:

```
if [ condition ]
then
    commands
fi
```

You may also write it on one line:

```
if [ condition ]; then
    commands
fi
```

Important keywords

Keyword	Purpose
if	Starts the condition
[]	Tests the condition
then	Starts commands executed when true
fi	Ends the if statement

Why fi?

fi is simply if written backwards.

```
if
↓
commands
↓
fi
```

3. How Bash Decides Whether the Condition Is True

Remember the previous lesson:

```
[ condition ]
```

is essentially the test command.

It produces an exit status:

```
0          → success / true
non-zero → failure / false
```

Therefore:

```
if [ "$x" -eq 10 ]
```

means:

Run the test

↓

Is x equal to 10?

↓

Exit status

↓

0 → execute then block

non-zero → don't execute then block

This is the connection between `test` and `if`.

4. `if` with Integer Conditions

Example:

```
if [ "$age" -ge 18 ]
then
    echo "You are an adult"
fi
```

Other common comparisons:

```
-eq → equal
-ne → not equal
-gt → greater than
-lt → less than
-ge → greater than or equal
-le → less than or equal
```

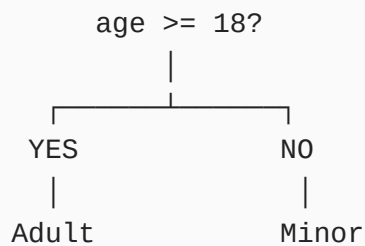
5. `else`

Sometimes you want to perform one action when the condition is true and another when it's false.

Use `else` .

```
if [ "$age" -ge 18 ]
then
    echo "Adult"
else
    echo "Minor"
fi
```

The logic becomes:



Mental model

`if` = "Do this if true."

`else` = "Otherwise, do this."

6. `elif`

What if there are **more than two possibilities**?

That's where `elif` comes in.

`elif` means:

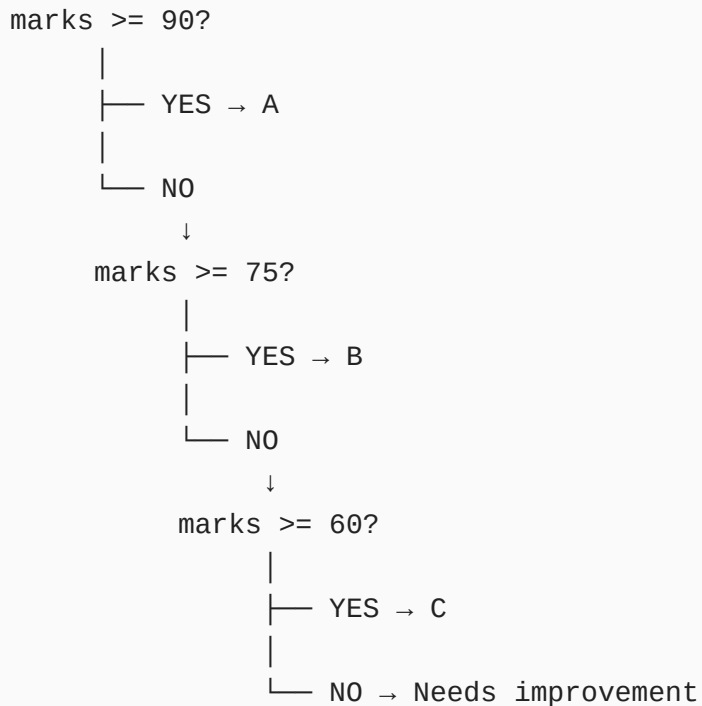
`else if`

Example:

```
if [ "$marks" -ge 90 ]
then
    echo "A"
elif [ "$marks" -ge 75 ]
then
    echo "B"
```

```
elif [ "$marks" -ge 60 ]
then
    echo "C"
else
    echo "Needs improvement"
fi
```

The script checks conditions from **top to bottom**.



Important

Once Bash finds a true condition in an `if/elif` chain, it executes that block and skips the remaining branches.

7. Multiple Conditions with `&&`

You can combine conditions using logical AND.

Conceptually:

```
if [ condition1 ] && [ condition2 ]
then
    commands
fi
```

Both conditions must be true.

Example:

```
if [ "$age" -ge 18 ] && [ "$country" == "India" ]
then
    echo "Condition satisfied"
fi
```

Logic:

```
age >= 18? — YES —┐
                    └─ YES → execute
country == India? —YES——┘
```

If either condition fails, the overall condition fails.

8. Multiple Conditions with `||`

`||` represents logical OR.

```
if [ condition1 ] || [ condition2 ]
then
    commands
fi
```

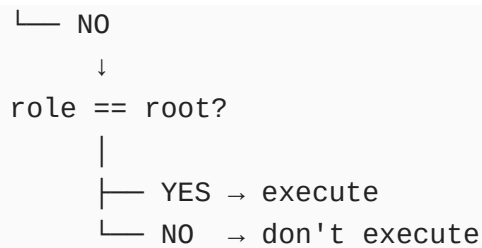
Only **one condition** needs to be true.

Example:

```
if [ "$role" == "admin" ] || [ "$role" == "root" ]
then
    echo "Privileged user"
fi
```

Logic:

```
role == admin?
|
└─ YES → execute
|
```



9. `if` + File Testing

This connects directly to the previous lesson's file operators.

For example:

```
if [ -d "$directory" ]
then
    echo "Directory exists"
else
    echo "Directory does not exist"
fi
```

Here:

```
-d
↓
Is this path a directory?
```

Other useful tests include:

```
-e → exists
-f → regular file
-d → directory
-r → readable
-w → writable
-x → executable
```

11. `case` Statements

The second major topic is the `case` statement.

A `case` statement is useful when you need to compare **one value against multiple possible patterns**.

Instead of:

```
if ...
elif ...
elif ...
elif ...
else ...
fi
```

you can often use:

```
case ...
...
esac
```

12. Basic `case` Syntax

General structure:

```
case "$variable" in
    pattern1)
        commands
        ;;
    pattern2)
        commands
        ;;
    pattern3)
        commands
        ;;
    *)
        default_commands
        ;;
esac
```

Important pieces:

Component	Purpose
case	Starts case statement
"\$variable"	Value being examined
in	Begins the patterns
pattern)	Defines a matching case
;;	Ends that case
*	Default/fallback pattern
esac	Ends case statement

Again:

```
case → esac
```

is similar to:

```
if → fi
```

13. Simple case Example

Suppose:

```
choice="start"
```

You can do:

```
case "$choice" in
    start)
        echo "Starting"
        ;;
    stop)
        echo "Stopping"
        ;;
    restart)
        echo "Restarting"
        ;;
esac
```

Bash checks:

```
choice = start?  
    ↓  
YES → Starting
```

It doesn't need to test the other cases once a match is found.

14. Why Use `case` ?

Imagine this with `if`:

```
if value == A  
else if value == B  
else if value == C  
else if value == D  
else if value == E
```

It can become difficult to read.

`case` makes the structure clearer:

```
case value in  
  A) ...  
  B) ...  
  C) ...  
  D) ...  
  E) ...  
esac
```

General rule

Use:

`if` when you're evaluating **conditions or ranges**.

Use:

`case` when you're matching **one value against several possible patterns**.

15. `case` Uses Patterns

One particularly important point:

`case` uses **shell pattern matching**, commonly called globbing patterns.

It does **not** use regular expressions by default.

For example:

```
case "$filename" in
    *.txt)
        echo "Text file"
        ;;
    *.log)
        echo "Log file"
        ;;
esac
```

Here:

```
*.txt
```

means a filename ending in `.txt`.

16. Common `case` Patterns

Exact match

```
start)
```

Matches exactly:

```
start
```

Wildcard

```
*.txt)
```

Matches names ending in `.txt`.

Multiple alternatives

You can combine patterns using `|`:

```
start|run)
    echo "Starting"
;;
```

This matches either:

```
start
```

or:

```
run
```

Single-character wildcard

```
file?.txt)
```

The `?` represents one character.

For example:

```
file1.txt
fileA.txt
fileX.txt
```

could match.

17. The `*` Default Case

A very important pattern is:

```
*)
```

This acts as the **default/fallback case**.

Example:

```
case "$choice" in
    start)
        echo "Starting"
        ;;
    stop)
        echo "Stopping"
        ;;
    *)
        echo "Invalid option"
        ;;
esac
```

If the user enters something other than `start` or `stop`, the `*` case catches it.

Why `*`?

Because in shell pattern matching, `*` can match essentially any string.

Therefore:

```
Known input → specific case
Unknown input → *
```

18. Why `*` Should Usually Be Last

Consider:

```
case "$choice" in
    *)
        echo "Default"
        ;;
    start)
        echo "Starting"
        ;;
esac
```

The `*` pattern can match `start`, so the later `start)` case will never be reached.

Therefore, conventionally:

```
case "$choice" in
    start)
        ...
        ;;
    stop)
        ...
        ;;
    *)
        ...
        ;;
esac
```

Mental model

Think of `*` as:

"If nothing else matched, come here."

19. Why Quote the Variable?

Use:

```
case "$choice" in
```

rather than unnecessarily leaving the expansion unquoted.

Similarly, in conditions:

```
[ "$name" == "Sourya" ]
```

Quoting variables is a good habit because it protects against unexpected whitespace and empty values during ordinary command/test parsing.

Important distinction

For `case`, the patterns themselves are intentionally **unquoted** when you want **pattern syntax**:

```
*.txt)
```

If you quote the pattern, you can change its meaning because the pattern is no longer treated in the same way.

20. `::` Is Important

Every case branch normally ends with:

```
::
```

Example:

```
case "$option" in
  1)
    echo "Option 1"
    ;;
  2)
    echo "Option 2"
    ;;
  *)
    echo "Invalid"
    ;;
esac
```

Think of `::` as:

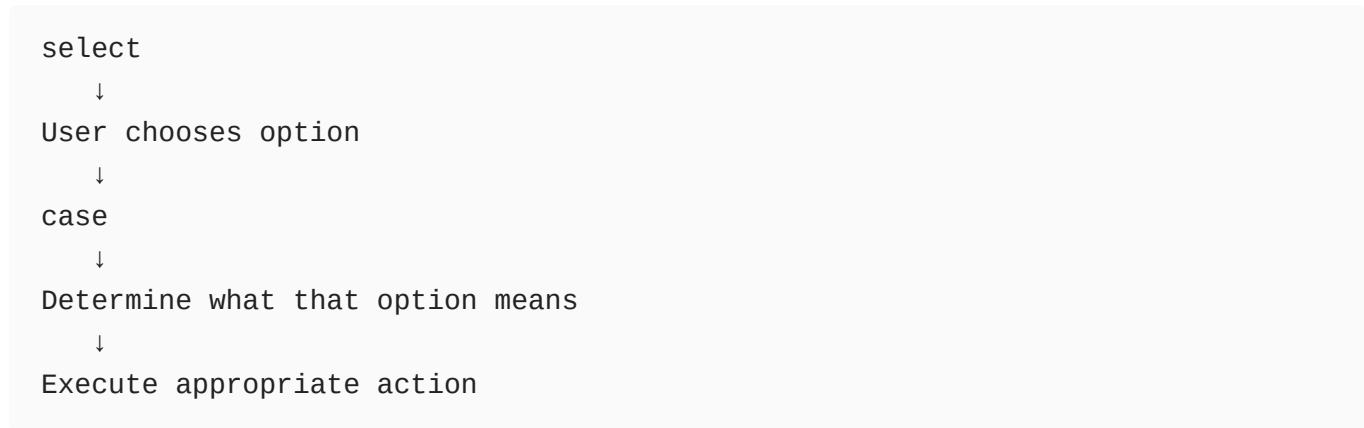
"This case block is finished."

Without the appropriate terminator, Bash cannot correctly parse the traditional case branch structure.

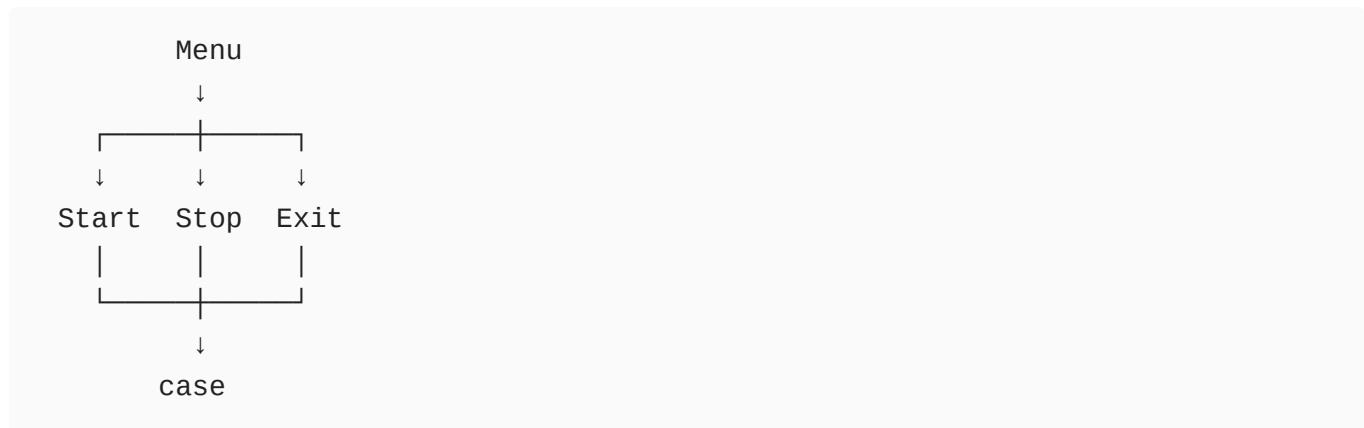
21. `case` + `select`

This is especially useful considering the previous lesson where you learned `select`.

You can combine them:



For example:



This is a very common pattern for interactive Bash scripts.

22. if vs case

Feature	if	case
Best for	Conditions	Multiple value/pattern matches
Ranges	Excellent	Not ideal
Boolean logic	Excellent	Less natural
Multiple exact choices	Can do	Excellent
Pattern matching	Limited	Excellent
Readability for menus	Can become messy	Very good

Feature	if	case
Default branch	else	*

Example: if

Good:

```
if [ "$age" -ge 18 ]
then
    echo "Adult"
fi
```

Because you're testing a numerical condition.

Example: case

Good:

```
case "$action" in
    start)
        ...
        ;;
    stop)
        ...
        ;;
    restart)
        ...
        ;;
esac
```

Because you're matching one variable against several choices.